Anni Alanen, Ira Kumpula, Nur Hana, Sandra Juursoo

Project Plan for Library Reservation System – MetBook

Metropolia Ammattikorkeakoulu

26.1.2026

Contents

1. Project Overview 1
2. Project Objectives 1
3. Scope and Deliverables 2
4. Project Timeline 3
5. Resource Allocation 4
5.1 Team Members and Roles 4
5.2 Technologies and Tools 5
6. Risk Management 5
7. Testing and Quality Assurance 6
8. Documentation and Reporting 7

1. Project Overview

Project Title:

Library Book Reservation System

Problem Summary:

Students and teachers at Metropolia University of Applied Sciences struggle to access library materials efficiently. Additionally, without reminder notifications, users often forget to return borrowed books on time, resulting in unnecessary late fees and inconvenience. This project aims to address these challenges by developing a library reservation system that simplifies access to learning materials and supports timely book returns.

Intended Audience/Users:

The primary users of the system are students and teachers at Metropolia University of Applied Sciences. Students will use the system to find and borrow books for their coursework and personal learning. Teachers will access academic resources for teaching and research purposes. Both user groups will have the same access rights and capabilities within the system.

Main Features/Components:

The main features of the project are:

- Browsing and searching the library book catalog by title or author
- Filtering books by publication year, language, and availability
- Checking book availability status
- Viewing detailed book information
- Reserving and borrowing books.
- User registration and login
- Viewing user profiles, active loans, reservations, and loan history
- Receiving reminders for due dates and overdue loans

2. Project Objectives

The key objective of this project is to design and implement a user-friendly library reservation system that provides core library functionalities, including book searching, browsing,

availability checking, reservations, and borrowing. The system will also enable users to register, log in, and manage their loans and reservations. In addition, an automated reminder system will be implemented to notify users of upcoming due dates and overdue books, helping to reduce late returns.

Another objective of the project is to apply Agile and DevOps practices throughout the project development process. This includes sprint-based development, version control using GitHub, and continuous integration. Automated testing will be integrated into the workflow to ensure software quality, and the application will be packaged and deployed using Docker containers.

3. Scope and Deliverables

This project aims to develop a library book reservation system using DevOps principles. The focus of this course is directed towards the process of building rather than the final product itself. Concentrating on implementing automated build, testing, and deployment pipelines as well as measuring code quality through unit testing. Therefore, the user interface of Library Book Reservation System remains simple, even though adding more complex features has been tempting.

In-Scope Items

- Basic user interface to browse, search and filter available books
- User registration
- User login & authentication
- Functionality to borrow, reserve and return books
- Receive notifications inside application
- Backend logic for handling book availability and user authentication (Java)
- Data storage solution (MariaDB)
- Unit testing (JUnit)
- Code coverage analysis
- Automated test execution (CI pipeline)
- UML & ER Diagrams
- Interface mockups (Figma)
- CI/CD pipeline implementation (Jenkins)

- Docker image creation and execution
- Environment-based configuration
- Well-maintained GitHub repository with README explaining pipeline and deployment process

Out-of-Scope

- Role-based features beyond standard user
- Admin portal
- Advanced library features (recommendations)
- Receive notifications outside of the application
- Change existing user information

4. Project Timeline

Development of this project is approached using the Agile and Scrum practices. Therefore, the project lifecycle has been divided into four sprints, which includes key phases and milestones of the project. Every sprint ends with sprint review with the stakeholders (teacher).

Sprint 1 (15.1.-29.1.): Project starts with requirement gathering, defining scope of the product as well as setting up GitHub-repository and Trello to keep track of the timeline. Next steps include technology selection, writing user stories, creating product vision and plan based on the earlier decisions. Next step in planning is creating wireframes on paper and designing application UI with Figma.

Sprint 2 (29.1.-12.2.): Starts with implementation of database. Which includes designing database schema, creation of tables/collections and CRUD testing. Next starts the development of user interface focusing on layout and interactive elements. In addition, sprint 2 includes writing unit tests for key backend and frontend functions as well as integrating JaCoCo to generate coverage reports.

Sprint 3 (12.2.-5.3.): The third sprint focuses on extending functional prototype by implementing more complex features, core functionality validation and optimization. As well as configuration and integration of Jenkins for CI/CD to automate the building, testing and

deployment processes. New features include user authentication, data validation and sanitization to prevent security vulnerabilities, enhanced search functionality with filtering and sorting options and reporting analytics dashboard for administrators. Validation of core functionalities is accomplished with end-to-end, performance and security testing. At the end of this sprint, product is ready for functional review, with all main features implemented and testable.

Sprint 4 (5.3.-12.3.): The last sprint includes: ensuring all planned features are fully implemented and operational through testing, containerization of the application (frontend and backend), Docker file creation, image building and local testing as well as publishing the docker image to docker hub. At the end of this project, presentation and functional prototype will be shared with class.

5. Resource Allocation

5.1 Team Members and Roles

The project has four members: Anni Alanen, Ira Kumpula, Nur Hana and Sandra Juursoo. All team members are needed to complete the project and its various tasks. There are different roles during the project, of which the lead role is the Scrum Master. The role will alternate as follows: during Sprint 1, Ira is the Scrum master; Sandra in Sprint 2, Anni in Sprint 3, and Nur in Sprint 4. The Scrum Master is responsible for making sure that every task planned for the Sprint is done on time. The Scrum Master also assigns tasks to the team members and gives appropriate deadlines for all tasks.

The project includes multiple phases and technologies, and all team members will participate in each phase to make sure that that everyone gets familiar with new technologies and tools. The team has gained experience from previous projects and has found it reasonable to include all members in all phases of the project. In a previous web application project, the team was divided strictly into backend and frontend teams. That caused conflicts when merging the project – the frontend team had no clue about backend code and vice versa. For this project, the responsibilities will be decided during each Sprint by the Scrum Master.

5.2 Technologies and Tools

The project's backend will be developed using Java and Spring Boot. The team is curious to learn new technologies, so Spring Boot was chosen even though none of the team members has used it before. For learning material, the team will use spring.io and geeksforgeeks.org websites.

The frontend will be implemented using React and Tailwind CSS. A prototype of the user interface will be created with Figma during the first sprint. MariaDB will be used as a database, and Docker will be used for deploying the application. The team will learn more about the Docker during the course.

6. Risk Management

There are several potential risks that may affect the success of the project. Each recognized risk is listed in Table 1, which includes the description of the risk, its likelihood of occurring (low, medium or high), impact on the project (low, medium or high) and strategy how to manage and reduce the risk.

Table 1. Potential risks for the project.

Risk	**Description**	**Likelihood**	**Impact**	**Strategy to manage**
Delay	The project is not finished before due date	Medium	High	The Scrum Master allocates the tasks of each sprint and sets due dates.
Tourism	A team member does not participate in the project	Low	High	Communicate often, inform about problems. The team has meetings at least once per week.
Wide topic	The project has a huge number of features, and the team doesn't have enough time to develop it fully.	Medium	Medium	Set limits on the project. Implement the most important features first.

Unfamiliar tools	The team does not learn how to use Docker or Spring Boot.	Low	High	Everyone reads learning material about Docker. The team asks for help from teacher.
Version control conflicts	Merging conflicts occur when two or more team members modify the same file	Medium	Medium	Use branches, commit often.
Integration issues	Backend and frontend do not integrate	Medium	High	Communicate often, ask for help, implement tasks precisely.

7. Testing and Quality Assurance

Testing will be carried out according to the Agile practice. As each component is implemented, related tests will be written and executed. This helps find issues early and ensures the application works correctly as development continues.

Unit testing will be used to verify that each individual component functions correctly on its own. Integration testing will be used to validate the interaction between multiple components, ensuring that they work together as intended. User acceptance testing will verify real user workflows across the application.

The criteria for success are that all tests pass, every program component is covered by tests and interactions between components are correctly validated. In addition, all critical user workflows must be covered by successful user acceptance tests. Any issues identified during testing must be located and resolved before the application is considered complete.

Several tools and frameworks will be used for implementing testing. JUnit will be used for backend unit and integration testing, while JaCoCo will be used to measure and report backend test coverage. Spring Boot will be used for backend integration testing, including database CRUD operations. Jest will be used for frontend unit and integration testing, and Playwright will be used for user acceptance testing by simulating real user interactions in a browser. Jenkins will be used to automate the execution of tests, and generate test and coverage reports as part of the CI pipeline.

8. Documentation and Reporting

The project progress will be documented and reported to the Scrum Master throughout the development process. Documentation plays an important role in clarifying and explaining the system and its functionality. Multiple documents will be produced during the project, some of which may include diagrams to better illustrate system structure and behavior. Some documentation will be intended for different audiences, with certain documents aimed at developers and others at end users.

The documentation produced during the project will include the project vision, project plan, sprint review reports, project presentation, JaCoCo coverage reports, and the GitHub README.

Project progress reporting will take place during biweekly meetings with the Scrum Master at the end of each sprint. During these meetings, the sprint review report will be used to review and discuss what the team has completed during the sprint. Sprint review reports will be produced by the end of sprints 1, 2, and 3. The sprint review report will include the following items: the sprint goal, completed user stories and tasks, a demo summary, what went well, what could be improved, and the focus for the next sprint. In addition, the report will include a table demonstrating the time spent by each team member during the sprint.